

Playwright API Automation Framework

Detailed Technical Guide

A practical map of the test architecture, execution paths, validation, reporting, and CI workflow.

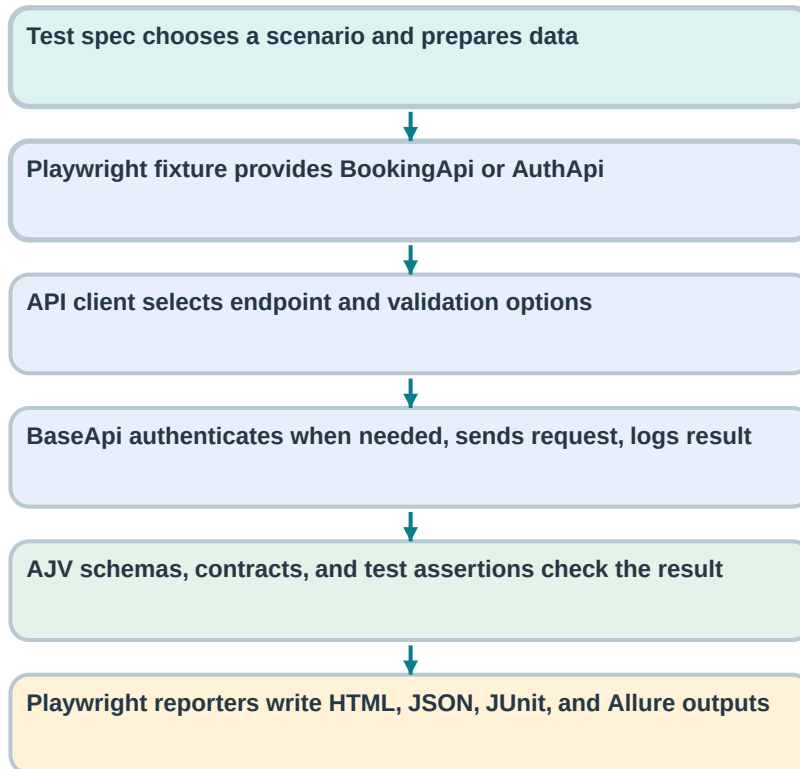
Inside this guide

- Architecture and request lifecycle
- Authentication and booking CRUD flows
- Configuration and run commands
- Schemas, contracts, assertions, and reports
- Docker and GitHub Actions pipeline
- Repository-specific notes and troubleshooting

1. Executive overview

This repository is a TypeScript API automation framework using Playwright Test. It sends API requests to a configured external booking service, validates both the HTTP result and the response data, and emits several report formats for local review and CI artifacts. It is a test client: it does not host or start the API being tested.

What happens during a typical test

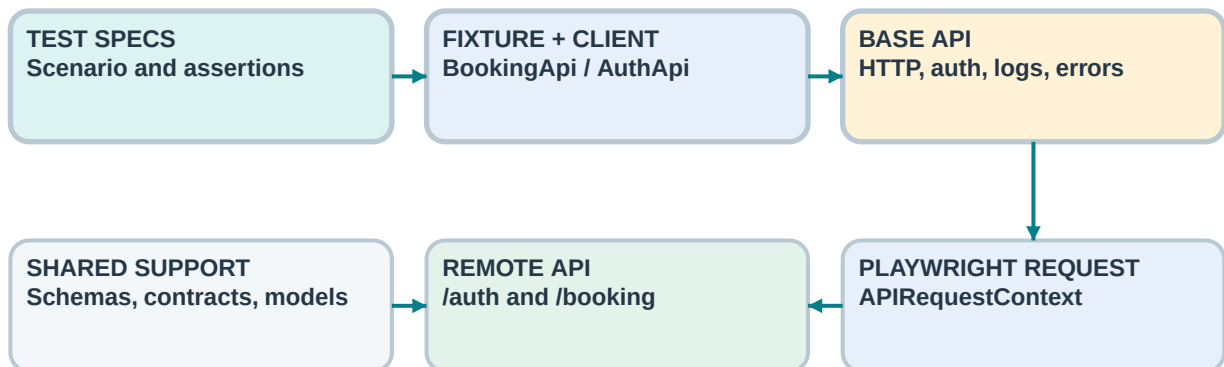


Key design ideas

- Tests express user-visible API behavior; client classes keep HTTP mechanics reusable.
- Factories make realistic data and deliberate invalid payloads without repeating object literals.
- Schemas check data shape while contracts and assertions check endpoint expectations and business values.
- Environment variables keep endpoints and credentials outside the test source.

2. Architecture and responsibilities

The layers have a clear direction: tests call the API clients, clients inherit shared transport behavior, and helpers provide cross-cutting validation and reporting. Models describe the values passed between layers.



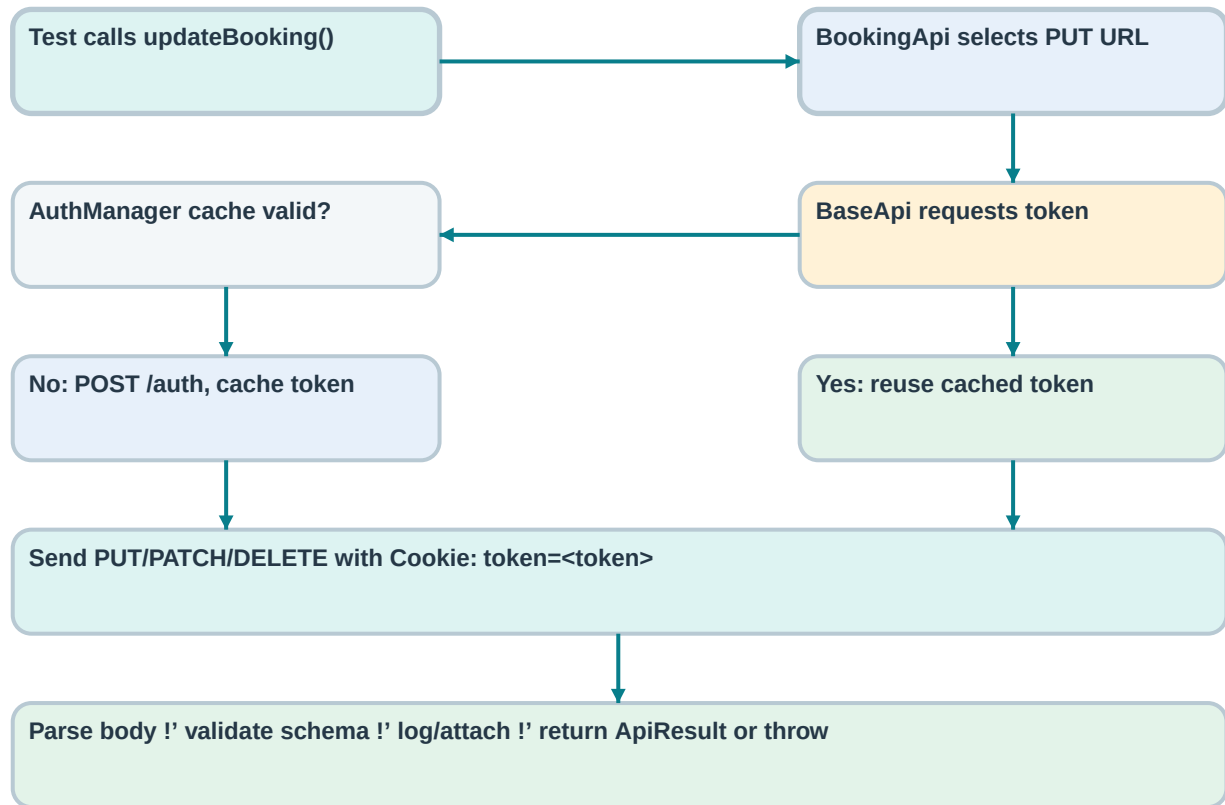
Responsibility map

- ``api/BaseApi.ts``: request execution, optional token header, payload/response schema checks, diagnostic logs, report attachments, expected-status handling, and one 401 retry.
- ``api/BookingApi.ts`` and ``api/AuthApi.ts``: endpoint-specific methods and default schemas.
- ``fixtures/apiFixture.ts``: constructs clients using Playwright's request fixture and labels Allure tests with the environment.
- ``helpers/``: reusable schema/contract validators, required-field checks, booking comparisons, API error formatting, console logs, and attachments.
- ``src/schemas/`` and ``src/contracts/``: machine-readable payload rules and endpoint-level expectations.
- ``models/``: compile-time shapes; they complement runtime schema validation rather than replacing it.

3. Request lifecycle and authentication

For an authenticated update, BaseApi first asks AuthManager for a token. AuthManager returns its cached token while it is younger than 30 minutes; otherwise it sends credentials to `/auth`. The protected request then carries the token in a Cookie header. A 401 clears the cache and triggers one retry.

Authenticated request sequence



Important behavior

GET and POST are unauthenticated by default in BaseApi. PUT, PATCH, and DELETE default to authentication. `updateBookingWithoutAuth` explicitly disables it for its 403 test. The token cache is static within the running Node process; independent workers/processes do not share it.

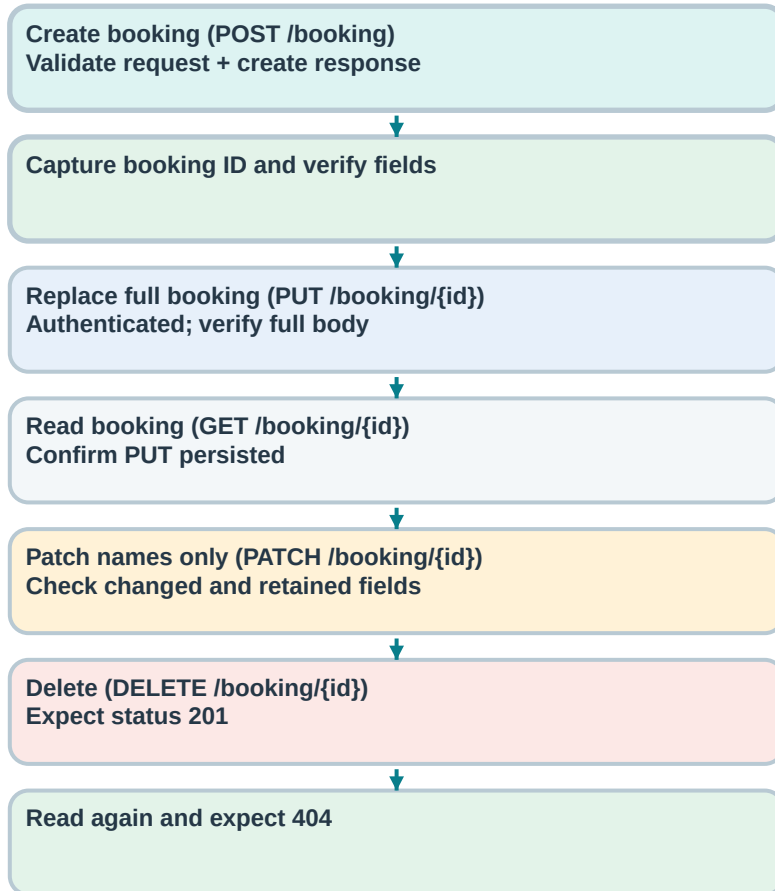
Response handling

Responses are parsed as JSON where possible, otherwise as text. A response schema is applied only when the HTTP response is successful. If a non-success status equals `expectedStatus`, the result is returned so the test can inspect it; otherwise BaseApi throws a diagnostic error containing method, URL, status, and body. A 401 retry is attempted only for requests marked as authenticated, and it happens once.

4. Booking scenarios as a lifecycle

The booking tests combine isolated checks with a stateful lifecycle. BookingFactory generates different data per run, then the test carries the booking ID returned by creation into the following operations.

Create !' replace !' patch !' delete

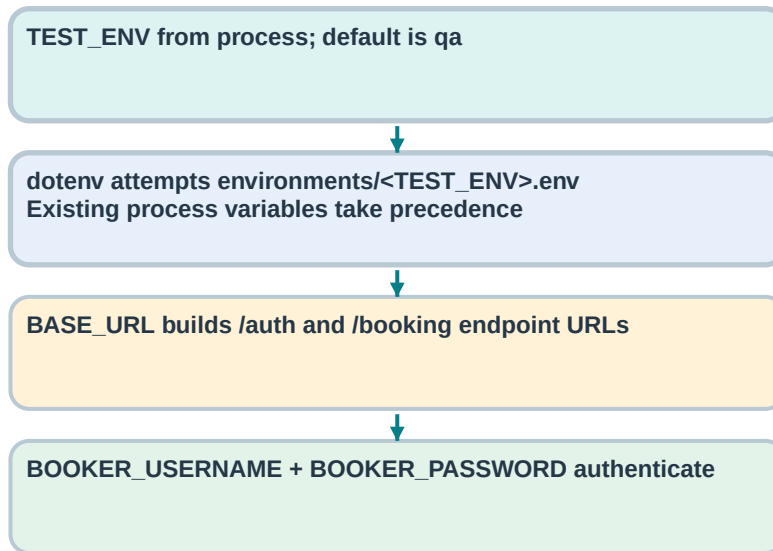


What the active specs cover

- Authentication: valid credentials return a nonempty token; invalid credentials are checked for HTTP 200 and `reason: Bad credentials`.
- Create: generated booking is posted, response fields and booking content are verified, then GET confirms persistence.
- Negative create validation: string price, non-boolean depositpaid, and unknown fields should fail local request schema validation before a network request.
- Missing booking: active GET case asks for a high nonexistent ID and expects 404. The positive GET example is currently commented out.
- CRUD lifecycle: checks full PUT replacement, PATCH partial update with other fields preserved, delete, and subsequent 404.
- Authorization: full update without auth is expected to return 403.

5. Configuration and running the suite

Configuration lookup



Expected variables

```
BASE_URL=https://restful-booker.herokuapp.com
BOOKER_USERNAME=admin
BOOKER_PASSWORD=password123
```

Repository observation

The reviewed repository tree does not include an environments/ directory. Supply the values in the shell, CI secrets, or a local environments/qa.env file. Do not commit real credentials.

Useful commands

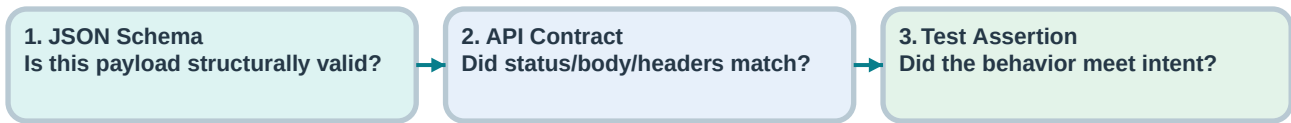
```
npm ci
npm run test:qa
npm run test:staging
npm run test:smoke
npm run test:regression
npx playwright test tests/api/updateBooking.spec.ts
npx playwright test --grep @smoke
```

Execution settings

Playwright discovers tests under `tests/api`. The configured project is Chromium and headless mode. Local execution permits up to four workers; CI uses one worker and retries twice. Trace is collected on the first retry, screenshots on failure, and video is configured on. The global expect timeout is four minutes. The configured reporters are HTML, list, JSON, JUnit, and Allure.

6. Validation: shape, contract, behavior

Three checks serve different purposes. Keeping them separate makes failures easier to interpret and helps avoid relying on TypeScript types for runtime guarantees.



Where each rule lives

- Request and response schemas are in `src/schemas/`; AJV with formats checks the runtime payload. `BaseApi` validates a provided request schema before sending, and validates a provided response schema only for successful HTTP responses.
- Contracts in `src/contracts/` capture expected status, response schema, and optional required headers. `validateContract` checks these against the ApiResult.
- Assertions in `helpers/apiAssertions.ts` compare status codes, required properties, booking field equality, and basic response conditions.
- `validateBookingPayload` adds explicit required-field and date-presence checks before the create scenario calls the API.

Request options and expected error

```
{
  requiresAuth?: boolean,
  expectedStatus?: number,
  requestSchema?: object,
  responseSchema?: object
}
```

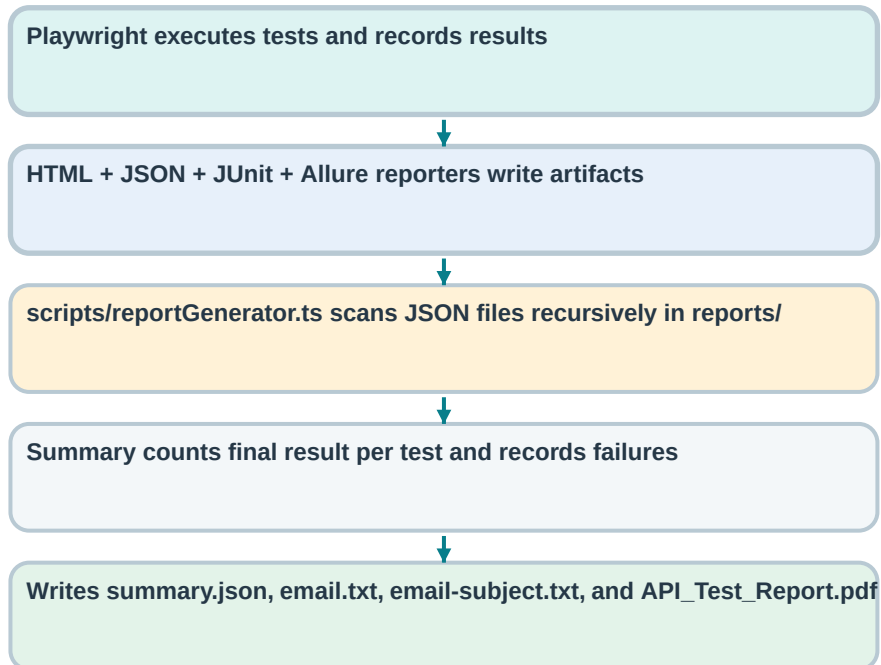
Error response pattern

For an expected 404, pass expectedStatus: 404 and the error response schema. This prevents BaseApi from throwing solely because the response is non-2xx, so the test can assert the returned status/body.

7. Reports and generated artifacts

Playwright writes each report during a test run. `REPORT_FILE`` selects the JSON report basename under `reports/``; `REPORT_FILE_XML`` can override the JUnit path. The HTML report folder in the current config is `playwright-report/``.

Reporting flow



Opening reports

```
npx playwright show-report playwright-report  
npm run allure:generate  
npm run allure:open  
npm run generate-report
```

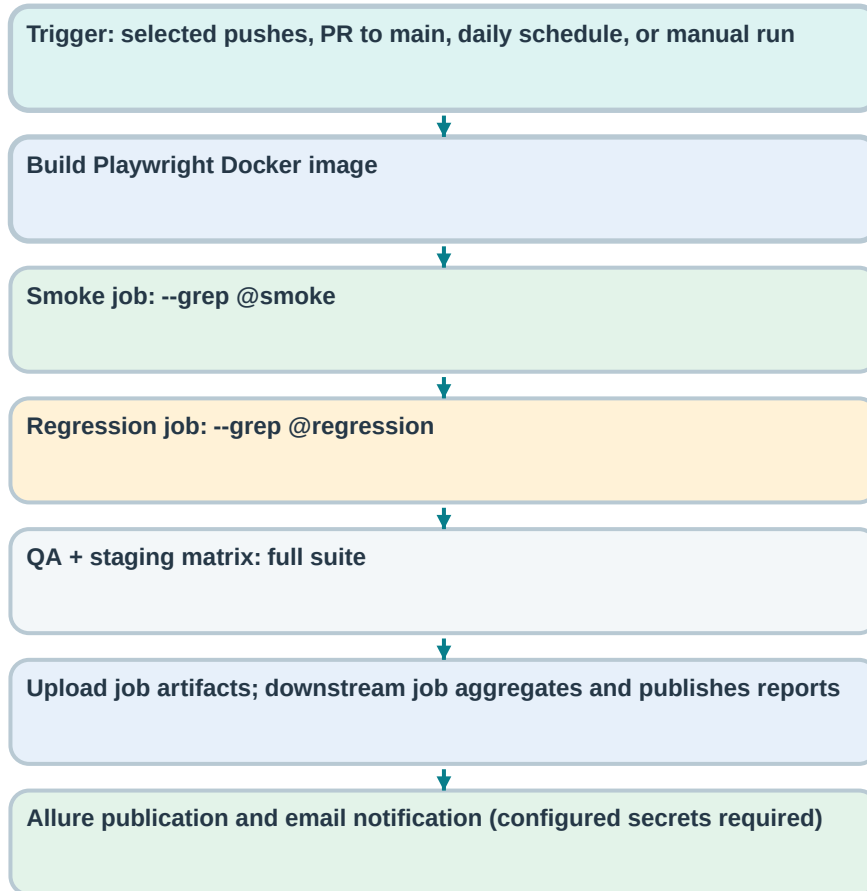
Input requirement

The summary generator expects Playwright JSON reports under `reports/`. If there are no JSON inputs, it exits with an error. A generated PDF from that script is a run summary, while this guide PDF documents the framework itself.

8. Docker and continuous integration

The Docker image packages the test runner, not the target API. The Compose service injects environment values, runs the test command, and mounts report folders so output remains available on the host.

CI pipeline map



Container quick start

```
docker build -t api-automation .  
docker compose run --rm -e TEST_ARGS="--grep @smoke" api-tests
```

Workflow dependencies

The report/publish job consumes artifacts from the test jobs and expects credentials and email-related GitHub secrets to be configured. Review artifact names and paths when maintaining the workflow. The workflow references additional report aggregation inputs and scripts; confirm they exist in the checked-out branch before relying on the publish stage.

9. Repository map and maintenance notes

Path	Purpose
api/	HTTP clients and common request behavior
tests/api/	Executable authentication and booking scenarios
fixtures/	Playwright fixture extension and client setup
helpers/	Validation, assertions, errors, logs, attachments
src/schemas/	AJV request/response JSON schemas
src/contracts/	Status, response shape, selected header expectations
src/factories/	Generated valid and intentionally invalid input
models/	TypeScript API data shapes
config/ + constants/	Environment loading and endpoint construction
scripts/	Aggregation/report generation
Docker + .github/workflows/	Container execution and CI automation

Repository-specific details to remember

- The implementation uses `BOOKER\_USERNAME` / `BOOKER\_PASSWORD`; some README examples use `USERNAME` / `PASSWORD`.
 - The README mentions some paths and run commands that differ from the current source tree and Playwright config. Prefer the checked-in code as the runtime reference.
 - The active positive GET-by-ID scenario is commented out; add or restore a positive retrieval check if that coverage is required.
 - Invalid login is asserted as a 200 response with an error reason. This matches the test's current expectation and should be kept aligned with the service behavior.
-

10. Troubleshooting and extension guide

Common failure paths

- Configuration error or malformed URL: confirm ``TEST_ENV``, ``BASE_URL``, and credentials are visible to the Node process. Remember dotenv does not overwrite existing process variables.
- 401 on update/patch/delete: verify credentials and token cookie format. AuthManager refreshes once after an authenticated 401; repeated failure is returned as an API error.
- Request schema error: the payload was rejected locally before HTTP. Compare the factory output to the schema and inspect AJV's reported field/path.
- Response schema or contract failure: inspect actual status/body/headers from the attached API log and compare with the endpoint schema/contract.
- No consolidated report: confirm at least one Playwright JSON report exists under ``reports/`` and uses the expected Playwright format.
- Docker outputs missing: verify the Compose bind mounts and that the container wrote into the mounted directories.

Adding a new endpoint scenario

